

Revisiting and Enhancing Graph Neural Networks through the Lens of Amortized Flows

Minjie Cheng

Gaoling School of Artificial Intelligence
Renmin University of China
Beijing, China
chengminjie@ruc.edu.cn

Dixin Luo

School of Computer Science and Technology
Beijing Institute of Technology
Beijing, China
dixin.luo@bit.edu.cn

Bokai Yan

Gaoling School of Artificial Intelligence
Renmin University of China
Beijing, China
2025001210@ruc.edu.cn

Hongteng Xu*

Gaoling School of Artificial Intelligence
Renmin University of China
Beijing, China
hongtengxu@ruc.edu.cn

Abstract

Model architecture design plays a central role in the study of Graph Neural Networks (GNNs), as it determines the capacity of GNNs and thus significantly impacts their practical performance. In this study, we propose a new plug-and-play module called Amortized Flow (AF) to enhance the capacity of GNNs through the lens of minimum-cost flow. In particular, for an arbitrary GNN model, we formulate its learning task in a framework based on the minimum-cost flow problem defined on the target graph. In this paradigm, the fitting of observed data/labels is achieved jointly through the message passing of the GNN and a minimum-cost flow defined on the graph edges. The amortization of the flow results in the proposed AF module, which avoids the time-consuming optimization of the flow. Adding the AF module as a residual block to the GNN leads to a new GNN architecture, whose number of additional parameters is independent with the graph size. Moreover, considering this flow-based module leads to a generalized framework for GNN design, which highlights the usefulness of edge features and graph incidence matrix. Experiments demonstrate that the AF module applies to different GNNs, and its application consistently improves model performance in various node-level classification tasks. The code is available at https://github.com/minjiecheng/GNN_AF.

CCS Concepts

• **Networks** → **Network architectures**; • **Computing methodologies** → **Learning paradigms**.

Keywords

Graph neural networks; Minimum-cost flow; Amortization; Node-level prediction

ACM Reference Format:

Minjie Cheng, Bokai Yan, Dixin Luo, and Hongteng Xu. 2026. Revisiting and Enhancing Graph Neural Networks through the Lens of Amortized Flows. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3774904.3792632>

1 Introduction

As a powerful tool of graph modeling, Graph Neural Network (GNN) has been widely used in many applications, such as social network analysis [13, 19, 45, 74], molecular modeling [30, 48, 60, 72], transportation forecasting [36, 59], recommendation systems [9, 11, 22], and so on. Typically, for a graph with node features, GNNs apply various message-passing mechanisms to embed the graph nodes and achieve node-level classification [41, 49] or regression [43, 61] based on the node embeddings. Given a subset of node labels, we often train GNNs in a node-level semi-supervised learning framework [10, 33, 65, 69], in which we optimize the node embeddings to predict the observed labels.

In the aspect of model architecture, existing GNNs can mainly be categorized based on their intrinsic message-passing mechanisms. The GNNs, such as Graph Convolution Network (GCN) [33, 42], Simplified Graph Convolution (SGC) [62], Graph Attention Network (GAT) [53], and their variants [21, 58, 76], apply fixed or learnable spatial convolutions/filters to achieve message-passing. These spatial GNNs are often correlated with graph spectral filters [4, 24]. For example, GCN [33] can be explained as a low-pass filter achieved by a first-order Chebyshev polynomial. Motivated by this observation, many efforts have been made to develop GNNs based on various polynomial filters, leading to a series of spectral GNNs, such as ChebNet [14] and its variants [27], GPR-GNN [12], and BernNet [28]. Besides approximated by the polynomial basis, the spectral GNNs can be learned by other strategies, e.g., Personalized PageRank in APPNP [23], graph optimization functions in GNN-LF/HF [75], ARMA filters [6], and diffusion kernel-based filters [16, 34, 65]. Recently, with the success of Transformer [52], some transformer-based GNNs have been proposed. In particular, Graphormer [70] and Nodeformer [63] consider dense message-passing operations based on multi-head attention maps. Specformer [7] and Polyformer [39] reformulate attention maps

*Hongteng Xu is the corresponding author of this work.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792632>

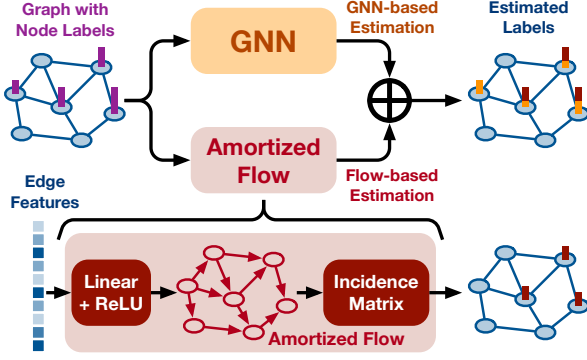


Figure 1: An illustration of the GNN with the proposed amortized flow module. Given a graph with partially observed node labels, we pass it through an arbitrary GNN model and our AF module, respectively. The AF module takes the edge features of the graph as input and estimates the flow on the graph edges by a simple neural network. Multiplying the flow with the incidence matrix of the graph leads to the flow-based estimations of the node labels. Adding the flow-based estimation to the GNN-based estimation leads to the final output of the model.

through the lens of spectral graph filtering. Besides the above GNNs based on spatial or spectral graph filtering, some new architectures have been explored based on graph kernels [51, 57] and factorization models [54, 66, 67], respectively.

In this study, we propose a plug-and-play module, called Amortized Flow (AF), to enhance arbitrary GNN architectures from the perspective of the minimum-cost flows on graphs [17, 18]. For an arbitrary GNN, we enhance it by considering the flow capturing the mass transport between the observed node labels and their GNN-based estimations. Learning a nonparametric flow with the GNN corresponds to the Quasi-Wasserstein learning method of the GNN [10], which introduces additional $O(E)$ parameters for a graph with E edges. To improve computational efficiency, we apply the neural network to achieve the amortization [1, 40, 50] of the flow. As illustrated in Figure 1, given a graph and an arbitrary GNN, the AF module takes its edge features as input and captures the flow on the graph edges by a simple neural network with extremely few parameters (e.g., a linear projection with a ReLU activation). Passing the flow through the incidence matrix [15] of the graph leads to a flow-based node label estimation. As a result, the aggregation of the GNN-based label estimation and the flow-based one leads to final estimation result. By learning an inductive neural network, we can estimate the flow directly, rather than relying on the time-consuming optimization [55, 66].

The AF module not only provides a new method to enhance GNNs but also provides a new perspective to revisit GNN architectures. We demonstrate the advantage of AF on representation power. Specifically, the GNN with our AF module can fit training data better than the classic GNN in theory. Moreover, we find that most existing polynomial-based spectral GNNs [12, 27, 28, 33, 73] can be viewed as the combination of a message-passing module

and an amortized flow module, which are implemented with various parameterization strategies. On the contrary, because of using edge features and an incidence matrix-based model architecture, there always exists a set of AF modules cannot be represented by existing GNNs. In practice, adding the proposed AF modules to these GNNs provides a new way to leverage node and edge features jointly, which helps improve model performance. Experiments demonstrate that the AF module is compatible with various GNNs, and adding it to the GNNs helps to improve their performance in various node-level classification tasks.

2 Related Work

Graph neural networks have matured rapidly from the early days of spatial/spectral graph convolution to hybrid message-passing models and, more recently, Transformer-style GNNs. Initially, GNNs often followed the spectral formulation [12, 14, 27, 28] or local smoothing [33, 62] idea, where each node aggregates neighbor features via normalized adjacency or graph Laplacian matrices. These designs set a foundational paradigm of neighborhood aggregation, leading to various polynomial-based spectral GNNs, e.g., ChebNet [14], APPNP [23], ChebNetII [27], BernNet [28], GPR-GNN [12], and so on. Later, spatial variants like GraphSAGE [26] introduced sampling and inductive frameworks, and attention-based models such as GAT [53] enabled learnable weighting of neighbor messages. Subsequent works improved depth [35], residual connections [25], polynomial basis [24], and topological rewiring to combat over-smoothing and over-squashing [2, 3, 47]. Recently, modern graph transformer approaches [7, 39, 63, 70] replace or augment the local aggregation with global attention modules over nodes, while injecting structural bias (via positional encodings, edge encoding, or learned adjacency) to maintain graph inductive priors [71]. These architectures allow more flexible long-range interaction modeling than pure message passing, which often achieve improved performance in node- and graph-level tasks.

The above GNNs are essentially based on various message-passing mechanisms. At the same time, non-message-passing graph learning models, including those based on kernels and graph factorization, have played an interesting complementary role. KerGNNs [20] integrates trainable hidden graphs with graph kernels to update node embeddings, achieving improved expressivity and interpretability compared to classic GNNs. KGNN [31] combines a kernel-based network and a GNN network, using memory of labeled graphs and explicit graph kernels to achieve graph retrieval. GBK-GNN [16] defines two kernels for homophily and heterophily and gates which kernel to apply per node pair, in effect selecting different similarity measures rather than fixed aggregation. Besides the above graph kernel-based models, some attempts have also been made to build graph factorization models. For example, GraphFM [64] incorporates factorization machine constructs inside graph representation learning to model high-order feature field interactions. Instead of simply summing neighbor features, it captures multiplicative (pairwise) interactions among features or among neighbors. Recently, some optimal transport-based graph representation models have been proposed, e.g., the Gromov-Wasserstein factorization model [66, 67] and the graph dictionary learning model [54]. These

non-message-passing GNNs are often applied to graph-level learning tasks.

Currently, some attempts have been made to design flow-based GNN architectures, e.g., the graph normalizing flow in [38], the Beltrami flow in [8], the FlowerFormer in [29], and the transductive GNN learned by a quasi-Wasserstein loss [10]. However, these models are either designed for graph generation tasks or too complex to handle large-scale node-level tasks. Unlike existing GNNs, our work develops an effective and efficient AF module to enhance arbitrary GNNs, resulting in a new GNN.

3 Proposed Method

3.1 Preliminaries

Denote a graph as $G(\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ represents the set of nodes and $\mathcal{E}$ represents the set of edges, respectively. We denote $\mathbf{d} = [d_v] \in \mathbb{N}^{|\mathcal{V}|}$ as the vector containing the node degrees. The nodes of the graph may have D -dimensional features, which are formulated as a matrix $\mathbf{X} \in \mathbb{R}^{|\mathcal{V}| \times D}$. In addition, we can define features on the graph edges, denoted as $\mathbf{Z} = [\mathbf{z}_e] \in \mathbb{R}^{|\mathcal{E}| \times K}$, where $\mathbf{z}_e \in \mathbb{R}^K$ denotes the K -dimensional feature of the edge $e = (u, v)$. We can set $\mathbf{z}_e$ as a scalar $d_u + d_v$ for undirected graph.¹

Typically, the graph G is associated with an *adjacency matrix* $\mathbf{A} \in \{0, 1\}^{|\mathcal{V}| \times |\mathcal{V}|}$, whose the non-zero elements indicate graph edges. The normalized adjacency matrix is $\tilde{\mathbf{A}} = \text{diag}(\mathbf{d})^{-\frac{1}{2}} \mathbf{A} \text{diag}(\mathbf{d})^{-\frac{1}{2}}$. Accordingly, the graph *Laplacian matrix* and its normalized version are $\mathbf{L} = \text{diag}(\mathbf{d}) - \mathbf{A}$ and $\tilde{\mathbf{L}} = \mathbf{I} - \tilde{\mathbf{A}}$, respectively. In addition, the *incidence matrix* of the graph is denoted as $\mathbf{S}_{\mathcal{V}} = [s_{ve}] \in \{0, \pm 1\}^{|\mathcal{V}| \times |\mathcal{E}|}$. For node v and edge e , the corresponding element in $\mathbf{S}_{\mathcal{V}}$ is

$$s_{ve} = \begin{cases} 1 & \text{if } v \text{ is "head" of edge } e \\ -1 & \text{if } v \text{ is "tail" of edge } e \\ 0 & \text{otherwise.} \end{cases} \quad (1)$$

When G is directed, the “head” and “tail” of each edge are predefined. When G is undirected, we can randomly define each edge’s “head” and “tail” in advance. According to the above definitions, we can build the connection among the three matrices above by the following equation:

$$\begin{aligned} \mathbf{S}_{\mathcal{V}} \mathbf{S}_{\mathcal{V}}^{\top} &= \text{diag}(\mathbf{d}) - \mathbf{A} = \mathbf{L}, \\ \underbrace{\text{diag}(\mathbf{d})^{-\frac{1}{2}} \mathbf{S}_{\mathcal{V}}}_{:= \tilde{\mathbf{S}}_{\mathcal{V}}} \underbrace{\mathbf{S}_{\mathcal{V}}^{\top} \text{diag}(\mathbf{d})^{-\frac{1}{2}}}_{:= \tilde{\mathbf{S}}_{\mathcal{V}}^{\top}} &= \mathbf{I} - \tilde{\mathbf{A}} = \tilde{\mathbf{L}}, \end{aligned} \quad (2)$$

where $\text{diag}(\mathbf{d})^{-\frac{1}{2}} \mathbf{S}_{\mathcal{V}}$ can be viewed as a normalized incidence matrix, which is denoted as $\tilde{\mathbf{S}}_{\mathcal{V}}$ for simplicity.

Suppose that a subset of nodes, denoted as $\mathcal{V}_L \subset \mathcal{V}$, are annotated with C -dimensional labels, i.e., $\mathbf{Y}_{\mathcal{V}_L} := \{\mathbf{y}_v \in \mathbb{R}^C\}_{v \in \mathcal{V}_L} \in \mathbb{R}^{|\mathcal{V}_L| \times C}$. We aim to learn a GNN, denoted as g , to predict the labels of the remaining nodes, i.e., $\{\mathbf{y}_v\}_{v \in \mathcal{V} \setminus \mathcal{V}_L}$. We often learn the GNN in the Maximum Likelihood Estimation (MLE) framework, which maximizes the conditional probability of all labeled nodes, i.e.,

$$\max_{\theta} \prod_{v \in \mathcal{V}_L} p(\mathbf{y}_v | \mathbf{X}, \mathbf{A}; \theta) \Leftrightarrow \min_{\theta} \sum_{v \in \mathcal{V}_L} \underbrace{\psi(g_v(\mathbf{X}, \mathbf{A}; \theta), \mathbf{y}_v)}_{= \hat{\mathbf{y}}_v}. \quad (3)$$

¹When the graph is directed, we can set $\mathbf{z}_e = [d_u^{\text{in}}, d_u^{\text{out}}, d_v^{\text{in}}, d_v^{\text{out}}]$, where d_v^{in} and d_v^{out} denote the in- and out-degrees of the node v , respectively.

Here, we denote the parameters of g as θ . Taking the adjacency matrix $\mathbf{A}$ and the node feature matrix $\mathbf{X}$ as input, the GNN g predicts the node labels. $g_v(\mathbf{X}, \mathbf{A}; \theta)$ estimates the label of node v , which is also denoted as $\hat{\mathbf{y}}_v \in \mathbb{R}^C$. Similarly, we can denote $g_{\mathcal{V}}(\mathbf{X}, \mathbf{A}; \theta) \in \mathbb{R}^{|\mathcal{V}| \times C}$ as the GNN-based label estimations for the node set $\mathcal{V}$. $\psi : \mathbb{R}^C \times \mathbb{R}^C \mapsto \mathbb{R}$ denotes the node-level loss function. In node-level classification tasks, the conditional probability $p(\mathbf{y}_v | \mathbf{X}, \mathbf{A}; \theta)$ is often modeled by the softmax function. Accordingly, we can implement the loss function ψ as the cross-entropy loss or the KL-divergence. In node-level regression tasks, the loss function can be the least-square error when $p(\mathbf{y}_v | \mathbf{X}, \mathbf{A}; \theta)$ is modeled as a Gaussian distribution.

As aforementioned, most existing methods implement the target GNN g by learnable spatial/spectral graph filters or Transformer-like architectures. Obviously, the model architecture is important, which determines the representation power of the GNN. In the following content, we design a new module to enhance an arbitrary GNN, which also helps us we revisit GNNs from the perspective of minimum-cost flow [17, 18].

3.2 Learning GNNs by Cost Flow Minimization

3.2.1 Minimum-Cost Flow Problem. Before introducing the proposed AF module, we first consider learning GNN in the framework of minimum-cost flow. Suppose that we have two measures on a graph $G(\mathcal{V}, \mathcal{E})$, denoted as $\boldsymbol{\mu} \in [0, \infty)^{|\mathcal{V}|}$ and $\boldsymbol{\gamma} \in [0, \infty)^{|\mathcal{V}|}$, respectively. Each element of a measure can be explained as the “mass” assigned to a node. Assume the two measures to be balanced, i.e., $\langle \boldsymbol{\gamma} - \boldsymbol{\mu}, \mathbf{1}_{|\mathcal{V}|} \rangle = 0$, where $\langle \cdot, \cdot \rangle$ is the inner product operator. We can transfer $\boldsymbol{\mu}$ to $\boldsymbol{\gamma}$ by the mass transport along the graph edges. The mass passing through the edge $e = (u, v)$ is called the *flow* on the edge, denoted as f_e . The summation of all the flows, i.e., $\|\mathbf{f}\|_1 = \sum_{e \in \mathcal{E}} |f_e|$, indicates the *transport cost* from $\boldsymbol{\mu}$ to $\boldsymbol{\gamma}$. The minimum-cost flow problem [17, 18] aims to find the optimal flows on the graph that minimize the transport cost from $\boldsymbol{\mu}$ to $\boldsymbol{\gamma}$, i.e.,

$$\min_{\mathbf{f}} \|\mathbf{f}\|_1, \text{ s.t. } \mathbf{f} \in \underbrace{\mathcal{U}^{|\mathcal{E}|} \cap \{\mathbf{f} \mid \mathbf{S}_{\mathcal{V}} \mathbf{f} = \boldsymbol{\gamma} - \boldsymbol{\mu}\}}_{\Omega(\mathbf{S}_{\mathcal{V}}, \boldsymbol{\mu}, \boldsymbol{\gamma})}, \quad (4)$$

where the feasible domain of the flow $\mathbf{f}$, denoted as $\Omega(\mathbf{S}_{\mathcal{V}}, \boldsymbol{\mu}, \boldsymbol{\gamma})$, consists of two constraints. The first constraint $\mathcal{U}^{|\mathcal{E}|}$ indicates the directions of the flows, where $\mathcal{U}$ is $[0, \infty)$ for directed graphs and is $\mathbb{R}$ for undirected graphs, respectively. It means that the flow of an edge can transport from “tail” to “head” in undirected graphs, which is not allowed in directed graphs. The second constraint $\mathbf{S}_{\mathcal{V}} \mathbf{f} = \boldsymbol{\gamma} - \boldsymbol{\mu}$ indicates that the flows are balanced, ensuring the transportation from $\boldsymbol{\mu}$ to $\boldsymbol{\gamma}$. By solving (4), we find the minimum-cost flow $\mathbf{f}^*$ and $\|\mathbf{f}^*\|_1$ is the corresponding minimum cost.

3.2.2 The Flow-based Learning Framework of GNN. In practice, we can extend the minimum-cost flow problem to more generalized scenarios by the following two modifications.

- **Vectorized Mass:** When the mass of each node is a vector rather than a scalar, the flow of each edge becomes a vector as well.
- **Valued Mass:** The mass of different nodes may have different values/significance. In such a situation, the optimal

flows should minimize the transport cost of the overall values rather than overall mass.

Take these two modifications into account. We can take the real label matrix Y_V and its GNN-based estimation $g_V(X, A; \theta)$ as two sets of vectorized mass defined on the graph $G(V, E)$. The values assigned to the nodes are set as the square root of node degrees in this study. Accordingly, the minimum-cost flow problem becomes

$$\begin{aligned} \min_F \|F\|_1, \\ \text{s.t. } F \in \mathcal{U}^{|\mathcal{E}| \times C} \cap \{F \mid S_V F = \text{diag}(\mathbf{d})^{\frac{1}{2}} (Y_V - g_V(X, A; \theta))\}, \end{aligned} \quad (5)$$

where $F = [f_e]$ is the flow matrix, and each $f_e \in \mathbb{R}^C$ indicates the vectorized flow on the edge e . Following the notation in (4), the feasible domain of F can be equivalently represented as

$$\begin{aligned} \Omega(\tilde{S}_V, g_V(X, A; \theta), Y_V) \\ = \mathcal{U}^{|\mathcal{E}| \times C} \cap \{F \mid \tilde{S}_V F = Y_V - g_V(X, A; \theta)\}. \end{aligned} \quad (6)$$

Here, the constraint $\tilde{S}_V F = Y_V - g_V(X, A; \theta)$ ensures the flows to capture the valued mass transport. Accordingly, $\|F\|_1 = \sum_{e \in \mathcal{E}} \|f_e\|_1$ measures the transport cost.

We can learn the GNN in the framework of minimum-cost flow. Given partially observed node labels, i.e., Y_{V_L} , we can learn the GNN together with the flow matrix by

$$\min_{F, \theta} \|F\|_1, \text{ s.t. } F \in \Omega(\tilde{S}_{V_L}, g_{V_L}(X, A; \theta), Y_{V_L}), \quad (7)$$

where the vector $\mathbf{d}_{V_L}$ contains the degrees of the nodes in V_L . Similarly, $\tilde{S}_{V_L}$ is a sub-matrix of the normalized incidence matrix, which only contains the rows of $\tilde{S}_V$ corresponding to the nodes in V_L . In this problem, the flow matrix and the GNN are optimized jointly to minimize the transport cost. This problem can be solved by various methods. For example, we can relax the strict constraint to a regularizer and solve the relaxed problem by Stochastic Gradient Descent (SGD) or its variants [32], or we can reformulate the problem in an augmented Lagrangian format and solve it by Bregman ADMM algorithm [10, 55].

Note that, when the node labels and their estimations are non-negative and the mass values are all-one vectors (i.e., $\text{diag}(\mathbf{d}_V) = \mathbf{I}$), the learning problem in (7) becomes the Quasi-Wasserstein (QW) loss in [10], which has achieved promising results in the learning tasks of various GNNs. However, in (7), the flow matrix is non-parametric, with $|\mathcal{E}|C$ learnable parameters. As a result, learning the flow matrix is time- and memory-consuming, preventing the QW loss from large-scale graph learning tasks. To overcome this challenge, we amortize the flow matrix via a simple neural network, which leads to the proposed AF module, and accordingly, new GNN architectures with merely additional $O(C)$ parameters.

3.3 Enhancing GNNs via Amortized Flows

3.3.1 Model Architecture. As aforementioned, the flow matrix F captures the transport from the GNN's output (i.e., $g_V(X, A; \theta)$) to the real label matrix Y_V . Accordingly, $\tilde{S}_V F$ corresponds to the residue between $g_V(X, A; \theta)$ and Y_V , denoted as Δ_V . To avoid iterative optimization of F , we apply the amortization strategy [1, 40, 50],

leveraging the following simple neural network as its surrogate:

$$\hat{\Delta}_V = \tilde{S}_V \underbrace{\sigma(ZW)}_{\text{Amortized Flow}}. \quad (8)$$

In this model, we takes the edge feature matrix Z as input, projects each edge feature to the C -dimensional space by a learnable projection $W \in \mathbb{R}^{K \times C}$, and obtains the activation of the projected result via ReLU (denoted as $\sigma(\cdot)$). As illustrated in Figure 1, the activations $\sigma(ZW_F)$ essentially represent amortized flows on graph edges: By learning the neural network to approximate the minimum-cost flow, we can replace the optimization of the flows with the feedforward computation of the neural network. Finally, multiplying the flow with the normalized incidence matrix leads to the estimated residue, denoted as $\hat{\Delta}_V$.

As illustrated in Figure 1, given an arbitrary GNN model g , we add our amortized flow module to it and estimate node labels as

$$\begin{aligned} \hat{Y}_V = g_V(X, A; \theta) + \hat{\Delta}_V = g_V(X, A; \theta) + \underbrace{\tilde{S}_V \sigma(ZW)}_{:= g'_V(X, A, Z, \tilde{S}_V; \theta')} \end{aligned} \quad (9)$$

The combination of the GNN and the AF module leads to a new GNN architecture, denoted as $g'_V(X, A, Z, \tilde{S}_V; \theta')$, whose parameters $\theta' = \{\theta, W\}$. Obviously, we just introduce KC additional parameters into the model. When the edge features are very simple, with $K = 1$, the number of additional parameters is just C .

3.3.2 Learning Algorithm. Given partially observed node labels Y_{V_L} , we leverage the MLE framework in (3) to learn the model. The corresponding optimization problem is

$$\min_{\theta'} \sum_{v \in V_L} \psi(g'_v(X, A, Z, \tilde{S}_v; \theta'), \mathbf{y}_v) + \beta \|\sigma(ZW)\|_1, \quad (10)$$

where the first term corresponds to the MLE of the model, and the loss function ψ is the Cross-Entropy (CE) loss for node-level classification and the Mean-Square-Error (MSE) for node-level regression, respectively. The second term penalizes the transport cost associated with the amortized flow. The trade-off between these two terms is controlled by the hyperparameter $\beta > 0$. In this work, we solve this problem iteratively by the Adam optimizer [32].

- **Remark 1.** The problem in (10) actually corresponds to a relaxed version of the learning problem in (7). In particular, when relaxing the constraint in (7) to a Bregman divergence-based regularizer, the problem becomes

$$\min_{F \in \mathcal{U}^{|\mathcal{E}| \times C}, \theta} \|F\|_1 + \lambda \underbrace{B_\phi(g_{V_L}(X, A; \theta) + \tilde{S}_{V_L} F, Y_{V_L})}_{= \sum_{v \in V_L} \psi(g_v(X, A; \theta) + \tilde{S}_v F, \mathbf{y}_v)}, \quad (11)$$

where $B_\phi(\cdot, \cdot)$ represents the Bregman divergence [5, 55] determined by the convex function ϕ . When ϕ is entropy (or ℓ_2 -norm), we can implement B_ϕ as the CE (or MSE) loss between the real and estimated labels, as shown in [10]. The weight of the Bregman divergence term is determined by the hyperparameter $\lambda > 0$. It is easy to find that when setting $\lambda = \frac{1}{\beta}$ and amortizing the flow matrix as $\sigma(ZW)$, the problem in (11) becomes the proposed learning problem in (10). Note that, such a relaxation is reasonable even necessary. When applying the amortized flow $\sigma(ZW)$, we may not be able to find the minimum-cost flow that guarantees $\hat{\Delta}_V = \Delta_V$. As a

result, the original constraint $\tilde{S}_V F = Y_V - g_V(X, A; \theta)$ may be infeasible, which has to be relaxed as a regularizer.

- **Remark 2.** In some cases, the edge feature Z can be unavailable. To make our model and its learning algorithm applicable in these cases, we can set $Z = \tilde{S}_V^\top X$, in which each edge feature corresponds to the residue between the feature of its head node and that of its tail node.

After obtaining the learned model $\{\theta^*, W^*\}$, we predict node labels by (9), i.e., $\hat{y}_v := g_v(X, A; \theta^*) + \tilde{S}_v \sigma(ZW^*)$ for $v \in V \setminus V_L$. The final estimation combines the GNN's output with the flow-based residue.

4 Advantages over Existing Methods

4.1 Data Fitting

As shown in (9), the GNN with our AF module leverages the node and edge features jointly to predict labels. Empirically, the prediction can often become more accurate because of introducing additional edge-level information. Moreover, in theory, we demonstrate that learning the new GNN model by our method (i.e., solving (10)) can fit training data better than traditional method does.

PROPOSITION 1. *Suppose that we learn various GNNs based on the graph $G(V, E)$ with adjacency matrix A , normalized incidence matrix $\tilde{S}_V$, node features X , edge features Z , and observed node labels Y_{V_L} . Let $\{\theta^*, W^*\}$ be the global optimal solution of (10) and $\theta^\ddagger$ be the global optimal solution of (3), where the loss ψ in (3) is determined by the Bregman divergence B_ϕ in (10). Then, we have*

$$B_\phi(g_{V_L}(X, A; \theta^*) + \tilde{S}_{V_L} \sigma(ZW^*), Y_{V_L}) \leq B_\phi(g_{V_L}(X, A; \theta^\ddagger), Y_{V_L}).$$

Proposition 1 is obvious — $\{\theta^\ddagger, 0_{K \times C}\}$ is a feasible solution of (10), so the corresponding objective is equal to or larger than that obtained by $\{\theta^*, W^*\}$. This proposition indicates that the GNN without our AF module can be treated as a special case having $W = 0_{K \times C}$. Learning θ and W jointly enhances the model capacity and representation power, leading to a better data fitting result.

- **Remark 3.** Because of introducing more information and learnable parameters, the GNN with AF module can fit training data better while suffering a higher risk of overfitting. Therefore, besides fitting training data, the learning problem in (10) further considers minimizing the transport cost $\|\sigma(ZW)\|_1$ as a part of the objective function. This term works as a regularizer suppressing the risk of overfitting.

4.2 Towards a Generalized Framework of GNNs

The proposed AF module provides a plug-and-play module compatible with arbitrary GNNs. Moreover, it provides us with a new perspective to revisit existing GNNs. Suppose that we have a graph $G(V, E)$ with a node feature matrix X and an edge feature matrix Z . We propose a generalized GNN framework as

$$Y = \text{MP}(X, A; \theta) + \text{Flow}(Z, \tilde{S}_V; \xi), \quad (12)$$

where the first term is the message-passing module based on the node feature matrix and the adjacency matrix, while the second term is the flow module based on the edge feature matrix and the (normalized) incidence matrix. Obviously, the g' in (9) is a special case of the above framework, in which $\text{MP}(X, A; \theta)$ corresponds to

an arbitrary existing GNNs, while $\text{Flow}(Z, \tilde{S}_V; \xi)$ is implemented by the AF module in (8).

It should be noted that the boundary between the message-passing module and the flow module is blurred in some special cases. In particular, we can construct edge features by $Z = \tilde{S}_V^\top X$, where each edge feature corresponds to the residue between the features of its head and tail nodes. In such a situation, some existing GNNs, which merely consider the message-passing of node features, can be revisited and reformulated based on the generalized framework in (12), with the help of the relations shown in (2). Table 1 shows the reformulations of some representative GNNs. A more general claim is given in the following proposition:

PROPOSITION 2. *Given an arbitrary M -order polynomial GNN, i.e., $Y = \sum_{m=0}^M \alpha_m \tilde{L}^m X$, where $\tilde{L}$ is the normalized graph Laplacian matrix and $\{\alpha_m\}_{m=0}^M$ are coefficients of the polynomial terms. Given $Z = \tilde{S}_V^\top X$, we can always reformulate the GNN by the generalized framework in (12). Specifically, $\forall i \in \{1, \dots, M-1\}$, we have*

$$Y = \underbrace{\sum_{m=0}^{i-1} \alpha_m \tilde{L}^m X}_{\text{MP module}} + \underbrace{\sum_{m=i}^M \alpha_m (\tilde{S}_V \tilde{S}_V^\top)^{m-1} \tilde{S}_V Z}_{\text{Flow module}}. \quad (13)$$

According to Proposition 2, many existing GNNs, especially those driven by spectral graph filters, can be viewed as special cases of the generalized framework. They reuse node features to derive edge features and, accordingly, make the flow module work like the message-passing module. As a result, the reasons for the limited performance of these GNNs might be *i)* the lack of informative edge features and *ii)* the “homogenization” of the message-passing and flow modules.

From this perspective, the advantage of our AF module is clear: Firstly, applying the AF module to an GNN can introduce informative edge features (that cannot be derived from the node features) into the model. Secondly, the AF module in (8) cannot be equivalently rewritten based on the adjacency or Laplacian matrix, whose architecture is different from that used for message-passing. Therefore, the AF module can enhance the representation power of the model and lead to a better GNN.

5 Experiments

To demonstrate the effectiveness of our AF module, we plug it into representative GNNs and learn the models in different node-level classification tasks. We compare the AF-assisted GNNs with the original ones learned by solving (3) and (7), respectively, in terms of their model performance and computational efficiency. All experiments are conducted on a machine equipped with eight NVIDIA RTX4090 GPUs. **Representative results are shown below, and more results can be found in Appendix.**

5.1 Implementation Details

5.1.1 Datasets. The datasets we considered consist of five homophilic graphs (i.e., **Cora**, **Citeseer**, **Pubmed** [49, 69], **Computers**, and **Photo** [41]) and five heterophilic graphs (i.e., **Chameleon**, **Squirrel** [46], **Actor**, **Texas**, **Cornell** [44], and **arXiv-year** [37]), respectively. We categorize the graphs according to the percentage of the edges connecting the nodes of the same class (i.e., the intra-edge rate) [56]. Each graph is unweighted and undirected.

Table 1: Revisiting some representative GNNs through the generalized framework in (12), under the assumption that $Z = \tilde{S}_V^T X$. For simplicity, we only show the filtering mechanism per layer for each model.

Model	Formulation per layer	MP($X, A; \theta$)	Flow($Z, \tilde{S}_V; \xi$)
GCN [33]	$(I + \tilde{A})X$	$2X$	$-\tilde{S}_V Z$
GPR-GNN [12]	$\sum_{m=0}^M \alpha_m \tilde{A}^m X$	$\sum_{m=0}^M \alpha_m X$	$\sum_{m=1}^M \alpha_m \sum_{j=1}^m \binom{m}{j} (-1)^j (\tilde{S}_V \tilde{S}_V^T)^{j-1} \tilde{S}_V Z$
BernNet [28]	$\sum_{m=0}^M \frac{\alpha_m}{2^M} \binom{M}{m} (2I - \tilde{L})^{M-m} \tilde{L}^m X$	$\frac{\alpha_0}{2^M} (I + \tilde{A})^M X$	$\sum_{m=1}^M \frac{\alpha_m}{2^M} \binom{M}{m} (2I - \tilde{S}_V \tilde{S}_V^T)^{M-m} (\tilde{S}_V \tilde{S}_V^T)^{m-1} \tilde{S}_V Z$

Table 2: Basic information of the homophilic graphs and the comparisons on node classification accuracy (%). For each task, the best result is bold. The column of “#Param.” shows the numbers of model parameters trained on the “Computers” graph.

Model	Method	#Param.	Cora	Citeseer	Pubmed	Computers	Photo	
	#Nodes ($ \mathcal{V} $)		2,708	3,327	19,717	13,752	7650	
	#Features (D)		1,433	3,703	500	767	745	
	#Edges ($ \mathcal{E} $)		5,278	4,552	44,324	245,861	119,081	
	Intra-edge rate		81.0%	73.6%	80.2%	77.7%	82.7%	
	#Classes (C)		7	6	3	10	8	
GCN	(3)	49,802	87.44 $\pm$ 0.96	79.98 $\pm$ 0.84	86.93 $\pm$ 0.29	88.42 $\pm$ 0.45	93.24 $\pm$ 0.43	—
GCN+F	(7)	2,508,412	87.88 $\pm$ 0.79	81.36$\pm$0.41	87.89$\pm$0.40	89.20 $\pm$ 0.41	93.81 $\pm$ 0.36	+0.83
GCN+AF	(10)	49,822	88.19$\pm$0.85	80.61 $\pm$ 0.56	87.84 $\pm$ 0.25	89.58$\pm$0.29	94.17$\pm$0.48	+0.88
GAT	(3)	399,390	89.20 $\pm$ 0.79	80.75 $\pm$ 0.78	87.42 $\pm$ 0.33	90.08 $\pm$ 0.36	94.38 $\pm$ 0.25	—
GAT+F	(7)	2,858,000	89.11 $\pm$ 0.66	80.19 $\pm$ 0.64	88.38$\pm$0.23	90.41 $\pm$ 0.28	94.65 $\pm$ 0.24	+0.18
GAT+AF	(10)	399,410	89.44$\pm$0.69	81.39$\pm$0.94	88.25 $\pm$ 0.22	90.62$\pm$0.35	94.67$\pm$0.30	+0.51
GIN	(3)	58,122	86.22 $\pm$ 0.95	76.18 $\pm$ 0.78	87.87 $\pm$ 0.23	80.87 $\pm$ 1.43	89.83 $\pm$ 0.72	—
GIN+F	(7)	2,516,732	86.24 $\pm$ 0.90	76.13 $\pm$ 1.09	87.53 $\pm$ 0.34	89.28$\pm$0.45	92.60$\pm$0.44	+2.16
GIN+AF	(10)	58,142	87.65$\pm$0.84	77.68$\pm$0.81	87.96$\pm$0.25	87.88 $\pm$ 0.59	92.25 $\pm$ 0.33	+2.49
GraphSAGE	(3)	99,530	88.24 $\pm$ 0.95	79.81 $\pm$ 0.80	88.14 $\pm$ 0.25	89.71 $\pm$ 0.38	95.08 $\pm$ 0.26	—
GraphSAGE+F	(7)	2,558,140	87.59 $\pm$ 0.77	80.52 $\pm$ 0.68	88.61 $\pm$ 0.32	90.17 $\pm$ 0.24	95.25 $\pm$ 0.25	+0.23
GraphSAGE+AF	(10)	99,550	88.34$\pm$0.74	80.71$\pm$0.70	88.78$\pm$0.16	90.60$\pm$0.45	95.49$\pm$0.34	+0.59
APNP	(3)	49,802	88.33 $\pm$ 0.77	81.28 $\pm$ 0.71	88.62 $\pm$ 0.33	86.27 $\pm$ 0.37	93.70 $\pm$ 0.27	—
APNP+F	(7)	2,508,412	88.74 $\pm$ 0.84	80.94 $\pm$ 0.61	89.48 $\pm$ 0.28	86.95 $\pm$ 0.82	94.43 $\pm$ 0.24	+0.47
APNP+AF	(10)	49,822	89.38$\pm$0.77	82.09$\pm$0.74	89.70$\pm$0.32	87.55$\pm$0.59	94.50$\pm$0.43	+1.00
BernNet	(3)	49,833	88.28 $\pm$ 1.00	79.81 $\pm$ 0.79	88.87 $\pm$ 0.38	87.61 $\pm$ 0.46	93.68 $\pm$ 0.28	—
BernNet+F	(7)	2,508,443	89.03$\pm$0.76	81.35$\pm$0.71	89.03 $\pm$ 0.38	89.58$\pm$0.47	94.55$\pm$0.39	+1.06
BernNet+AF	(10)	49,853	88.60 $\pm$ 0.71	81.27 $\pm$ 0.70	89.37$\pm$0.47	87.53 $\pm$ 0.45	93.80 $\pm$ 0.41	+0.46
ChebNetII	(3)	49,813	88.26 $\pm$ 0.89	80.00$\pm$0.74	88.57 $\pm$ 0.36	86.58 $\pm$ 0.71	93.50 $\pm$ 0.34	—
ChebNetII+F	(7)	2,508,423	88.54 $\pm$ 0.76	79.47 $\pm$ 0.70	89.47$\pm$0.36	90.43 $\pm$ 0.22	94.84$\pm$0.37	+1.17
ChebNetII+AF	(10)	49,833	88.64$\pm$0.81	79.99 $\pm$ 0.64	89.34 $\pm$ 0.40	90.46$\pm$0.39	94.75 $\pm$ 0.43	+1.25

5.1.2 GNN Architectures. In the following experiments, the models we considered include *i*) the representative spatial GNNs, i.e., GCN [33], GAT [53], GIN [68], GraphSAGE [26], and GCN-LPA [56] that combines the GCN with the label propagation algorithm; *ii*) state-of-the-art spectral GNNs, i.e., APPNP [23], BernNet [28], and ChebNetII [27]; and *iii*) the state-of-the-art graph transformer, PolyFormer [39]. For a fair comparison, we set the architectures of the models based on the code provided by the corresponding references and configure the algorithmic hyperparameters by grid search. More details of the hyperparameter settings are in Appendix.

5.1.3 Learning Tasks and Evaluation Measurements. For each graph, by default, the split ratio of labeled nodes is 60% for training, 20% for validation, and 20% for testing, respectively. In each learning task, we consider learning three models: *i*) learning a GNN by the traditional learning method in (3), *ii*) learning a GNN with a non-parametric flow matrix (GNN+F) by minimizing the QW loss [10]

in (7), and *iii*) learning a GNN with our AF module (GNN+AF) by solving (10), where we set $\text{diag}(\mathbf{d}_V) = \mathbf{I}$ to maintain consistency with the experimental setup of the QW loss method, and simply initialize the edge features by sampling from the Kaiming uniform distribution. For each model, we record its average performance and standard deviation in ten trials.

5.2 Numerical Comparisons

5.2.1 Node Classification. Tables 2 and 3 show the node classification results of spatial and spectral GNNs on the homophilic and heterophilic graphs, respectively. In each table, the last column records the overall improvements of different models compared to the baselines. The experimental results demonstrate the usefulness of our AF module — learning our AF module to GNNs helps improve model performance in most situations and leads to consistent overall improvements on both homophilic and heterophilic graphs. In particular, for the state-of-the-art spectral GNNs like

Table 3: Basic information of the heterophilic graphs and the comparisons on node classification accuracy (%). For each task, the best result is bold. The column of “#Param.” shows the numbers of model parameters trained on the “Squirrel” graph.

Model	Method	#Param.	Squirrel	Chameleon	Actor	Texas	Cornell	Arxiv-year	
#Nodes ($ \mathcal{V} $)			5,201	2,277	7,600	183	183	169343	
#Features (D)			2,089	2,325	932	1,703	1,703	128	
#Edges ($ \mathcal{E} $)			198,423	31,371	26,659	279	277	1166243	
Intra-edge rate			22.2%	23.0%	21.8%	6.1%	12.3%	22.0%	
#Classes (C)			5	5	5	5	5	5	
GCN	(3)	134,085	46.55 $\pm$ 1.15	63.57 $\pm$ 1.16	34.00 $\pm$ 1.28	77.21 $\pm$ 3.28	61.91 $\pm$ 5.11	44.40 $\pm$ 0.16	—
GCN+F	(7)	1,125,850	52.62 $\pm$ 0.49	68.10 $\pm$ 1.01	38.09 $\pm$ 0.50	84.10 $\pm$ 2.95	84.26 $\pm$ 2.98	44.70 $\pm$ 0.30	+7.37
GCN+AF	(10)	134,095	53.40$\pm$1.35	68.25$\pm$1.01	38.77$\pm$0.70	89.51$\pm$1.64	87.02$\pm$2.98	44.72$\pm$0.36	+9.00
GAT	(3)	1,073,679	48.20 $\pm$ 1.67	64.31 $\pm$ 2.01	35.68 $\pm$ 0.60	80.00 $\pm$ 3.11	68.09 $\pm$ 2.13	44.21 $\pm$ 0.30	—
GAT+F	(7)	2,065,444	55.03$\pm$1.35	67.35 $\pm$ 1.42	33.86 $\pm$ 2.13	80.33 $\pm$ 1.80	70.21 $\pm$ 2.13	44.41$\pm$0.28	+1.78
GAT+AF	(10)	1,073,689	51.47 $\pm$ 1.06	67.40$\pm$1.27	35.94$\pm$0.78	80.82$\pm$2.63	70.64$\pm$2.55	44.24 $\pm$ 0.25	+1.67
GIN	(3)	142,405	39.11 $\pm$ 2.23	64.29 $\pm$ 1.51	32.37 $\pm$ 1.56	72.79 $\pm$ 4.92	62.55 $\pm$ 4.80	44.39 $\pm$ 0.28	—
GIN+F	(7)	1,134,170	65.29$\pm$0.68	73.26$\pm$1.12	32.32 $\pm$ 1.93	77.54 $\pm$ 2.60	64.04 $\pm$ 3.62	44.56$\pm$0.26	+6.92
GIN+AF	(10)	142,415	50.76 $\pm$ 1.17	71.25 $\pm$ 1.45	34.43$\pm$1.03	78.69$\pm$3.11	67.87$\pm$5.96	44.51 $\pm$ 0.30	+5.34
GraphSAGE	(3)	268,101	43.79 $\pm$ 0.59	63.26 $\pm$ 1.09	38.99 $\pm$ 0.85	90.00 $\pm$ 2.30	84.26 $\pm$ 2.98	42.58 $\pm$ 0.18	—
GraphSAGE+F	(7)	1,259,866	54.37$\pm$0.89	68.32$\pm$0.68	37.82 $\pm$ 0.45	90.33$\pm$1.97	86.38$\pm$2.13	42.63 $\pm$ 0.21	+2.82
GraphSAGE+AF	(10)	268,111	53.09 $\pm$ 0.78	67.05 $\pm$ 0.94	40.02$\pm$0.56	90.16 $\pm$ 2.30	86.17 $\pm$ 2.98	42.74$\pm$0.23	+2.72
APNP	(3)	134,085	36.15 $\pm$ 0.75	52.93 $\pm$ 1.71	40.46 $\pm$ 0.64	91.31 $\pm$ 1.97	87.66 $\pm$ 2.13	41.05$\pm$0.32	—
APNP+F	(7)	1,125,850	38.73$\pm$1.06	53.76 $\pm$ 1.25	40.78$\pm$0.74	91.48$\pm$2.30	87.87$\pm$2.34	40.98 $\pm$ 0.28	+0.67
APNP+AF	(10)	134,095	37.81 $\pm$ 1.52	53.85$\pm$1.44	40.61 $\pm$ 0.74	91.48$\pm$1.97	86.81 $\pm$ 2.77	40.99 $\pm$ 2.77	+0.33
BernNet	(3)	134,106	51.15 $\pm$ 1.09	67.96 $\pm$ 1.05	40.72 $\pm$ 0.80	93.28 $\pm$ 1.48	90.21 $\pm$ 2.35	41.36$\pm$0.44	—
BernNet+F	(7)	1,125,871	55.22$\pm$0.64	71.66$\pm$1.18	40.91 $\pm$ 0.71	93.44$\pm$1.80	90.85$\pm$2.34	41.34 $\pm$ 0.37	+1.46
BernNet+AF	(10)	134,116	50.51 $\pm$ 1.15	70.59 $\pm$ 1.01	41.70$\pm$1.14	90.98 $\pm$ 1.97	90.64 $\pm$ 2.55	41.30 $\pm$ 0.39	+0.17
ChebNetII	(3)	134,096	57.78 $\pm$ 0.84	71.71 $\pm$ 1.40	40.70 $\pm$ 0.77	92.79 $\pm$ 1.48	88.94 $\pm$ 2.78	48.60 $\pm$ 0.17	—
ChebNetII+F	(7)	1,125,861	60.55$\pm$0.64	74.05$\pm$0.68	41.37$\pm$0.67	93.93 $\pm$ 0.98	87.23 $\pm$ 3.62	48.82 $\pm$ 0.19	+0.91
ChebNetII+AF	(10)	134,106	56.81 $\pm$ 0.95	73.41 $\pm$ 0.74	41.07 $\pm$ 1.06	94.10$\pm$1.48	89.15$\pm$2.77	49.06$\pm$0.31	+0.51

BernNet [28] and ChebNetII [27], learning with our AF module can improve their overall performance on both homophilic and heterophilic graphs consistently, resulting in the best performance in this experiment. For the simple GCN model [33], learning with our AF module improves its performance significantly and reduces the gap between its classification accuracy and that of the state-of-the-art models [23, 27, 28], especially in the experiments on heterophilic graphs.

In addition, Table 4 shows the effectiveness of our AF module on PolyFormer [39]. Here, we consider two PolyFormer backbones, which are based on monomial polynomials and Chebyshev polynomials, respectively. For each of these two PolyFormers, we can find that considering learnable flows helps improve the model performance in most situations, and the performance of our AF module is comparable to that achieved by learning nonparametric flows. In other words, our AF module is not only helpful for classic GNNs, but also improves the performance of the state-of-the-art graph Transformer.

5.2.2 Computational Efficiency and Scalability. As shown in Tables 2–4, compared to the GNNs with nonparametric flows, the GNNs with our AF module can achieve competitive learning results with much fewer parameters. This result verifies our claim — the amortized flow helps reduce parameters and memory costs while maintaining strong model prediction power. Figure 2 further visualizes various models on their training time per epoch. Compared to the baseline model, introducing learnable nonparametric

Table 4: Comparisons based on PolyFormer. For each graph, the best result is bold.

Model	Method	Homophilic graphs		Heterophilic graphs	
		Citeseer	Pubmed	Chameleon	Squirrel
PolyFormer _{Mono}	(3)	82.02 $\pm$ 0.82	90.08 $\pm$ 0.48	46.85 $\pm$ 1.74	42.41 $\pm$ 1.06
PolyFormer _{Mono} +F	(7)	82.05 $\pm$ 0.70	91.26$\pm$0.22	44.65 $\pm$ 2.13	42.92$\pm$0.90
PolyFormer _{Mono} +AF	(10)	82.63$\pm$0.89	91.26$\pm$0.21	47.00$\pm$2.02	42.39 $\pm$ 1.21
PolyFormer _{Cheb}	(3)	81.80 $\pm$ 0.83	90.67 $\pm$ 0.28	43.94 $\pm$ 2.42	41.83 $\pm$ 1.19
PolyFormer _{Cheb} +F	(7)	81.71 $\pm$ 0.78	90.99 $\pm$ 0.37	45.73 $\pm$ 3.14	42.12$\pm$1.58
PolyFormer _{Cheb} +AF	(10)	81.81$\pm$0.75	91.08$\pm$0.32	46.48$\pm$2.73	41.16 $\pm$ 1.01

or amortized flows leads to a mild increase of training time. In summary, while improving model performance significantly, learning our AF module only introduces few parameters and tolerable computational costs.

5.2.3 Distribution of Learned Flow. Figure 3 visualizes the histograms of the optimal flows learned by (10), i.e., $\sigma(\mathbf{ZW}^*)$. The backbone GNN used in this experiment is GCN [33] and the training graphs include a homophilic graph (“Computers”) and a heterophilic graphs (“Squirrel”). We can find that the amortized flows obey exponential distributions because of the usage of ReLU. It is reasonable from the perspective of optimization — the term $\|\sigma(\mathbf{ZW})\|$ can be explained as an exponential prior imposed on the flow matrix, which encourages the sparsity of the flows. In addition, we can find that the dynamic range of the learned flows

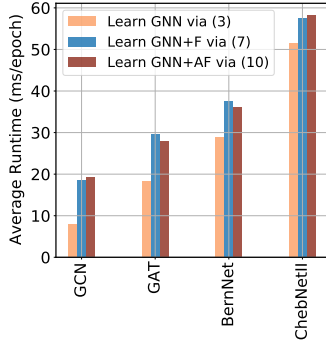


Figure 2: The runtime of training different models on the Photo graph.

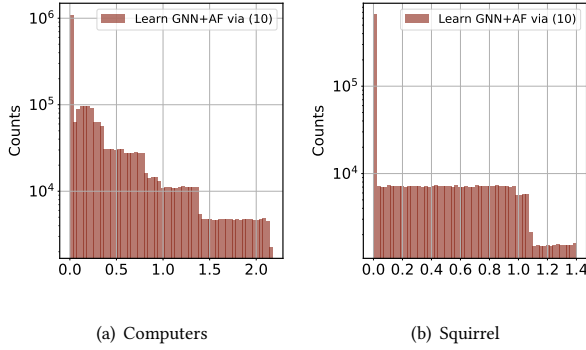


Figure 3: The histogram of learned flows for different graphs.

is small. This phenomenon implies that when learning flows with GNNs, the model tends to fit observed labels mainly by the GNN module and applies the flows to capture and regular the residue between the real and estimated labels, which matches the modeling principle shown in (8).

5.3 Analysis Experiments

5.3.1 Robustness to Label Insufficiency Issue. The flow associated with each GNN captures the label transport on graphs, whose feasible domain is determined by the observed training labels. The more labels we observed, the smaller the feasible domain is. To demonstrate the robustness of our method to the label insufficiency issue, we evaluate the performance of our method given different amounts of training labels. We train ChebNetII [27] (without or with flows) on four graphs. For each graph, we use $K\%$ nodes' labels to train the ChebNetII, where $K \in [1, 60]$, and apply 20% nodes for testing. Figure 4 shows that learning GNNs with flows leads to higher classification accuracy even when only 20% nodes or fewer are labeled. Moreover, the performance of AF module can be better than that of nonparametric flow, which means that the amortization strategy we used helps improve model capacity while suppressing the risk of over-fitting. The analysis experiment on the robustness to hyperparameter λ is provided in Appendix.

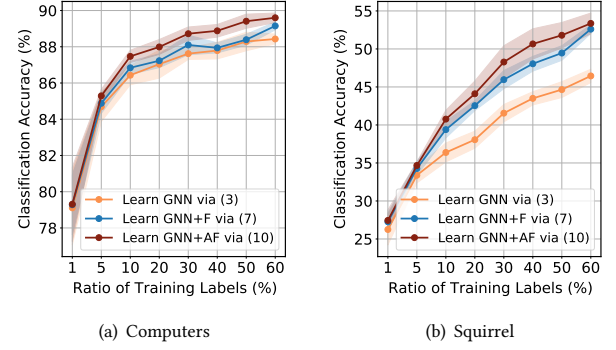


Figure 4: Illustrations of the learning methods' performance given different amounts of labeled nodes.

5.3.2 Edge feature setting analysis. When the edge features are sampled from the Kaiming uniform distribution, the AF module essentially estimates the label errors and acts as a loss function to guide GNN learning. Eventually, the GNN module dominates the model, and the AF module gradually converges to zero with training.

6 Conclusion and Future Work

We have proposed an amortized flow module for enhancing graph neural networks, which leads to a new flow-assisted GNN architecture with a corresponding framework based on minimum-cost flow. The proposed model achieves encouraging performance consistently in various node classification tasks while maintaining computational efficiency. Moreover, it provides us with a new perspective to revisit existing GNNs, leading to a generalized framework for GNN design. In this framework, we show the necessity of leveraging edge features and designing incidence matrix-based architecture that cannot be equivalently represented by graph adjacency/Laplacian matrix.

Limitations and Future Work. Empirically, we find that applying the ReLU activation helps improve model performance, while the nonnegative flows may be too strict for undirected graphs in the context of minimum-cost flow. How to instantiate the AF module with a more powerful and more reasonable neural network is left as our future work. In addition, we will consider developing new graph Transformers in the framework of minimum-cost flow.

Acknowledgements

This work was supported by NSFC 92270110 and the Fundamental Research Funds for the Central Universities. We also acknowledge the support provided by the fund for building world-class universities (disciplines) of Renmin University of China and by the funds from Beijing Key Laboratory of Research on Large Models and Intelligent Governance, Engineering Research Center of Next-Generation Intelligent Search and Recommendation, Ministry of Education, and from Intelligent Social Governance Interdisciplinary Platform, Major Innovation & Planning Interdisciplinary Platform for the "Double-First Class" Initiative, Renmin University of China.

References

- [1] Abhinav Agrawal and Justin Domke. 2021. Amortized variational inference for simple hierarchical models. *Advances in Neural Information Processing Systems* 34 (2021), 21388–21399.
- [2] Singh Akansha. 2025. Over-squashing in graph neural networks: A comprehensive survey. *Neurocomputing* (2025), 130389.
- [3] Uri Alon and Eran Yahav. 2021. On the Bottleneck of Graph Neural Networks and its Practical Implications. In *International Conference on Learning Representations*.
- [4] Muhammet Balçilar, Renton Guillaume, Pierre Héroux, Benoit Gaüzère, Sébastien Adam, and Paul Honeine. 2021. Analyzing the expressive power of graph neural networks in a spectral perspective. In *International Conference on Learning Representations*.
- [5] Jean-David Benamou, Guillaume Carlier, Marco Cuturi, Luca Nenna, and Gabriel Peyré. 2015. Iterative Bregman projections for regularized transportation problems. *SIAM Journal on Scientific Computing* 37, 2 (2015), A1111–A1138.
- [6] Filippo Maria Bianchi, Daniele Grattarola, Lorenzo Livi, and Cesare Alippi. 2021. Graph neural networks with convolutional arma filters. *IEEE transactions on pattern analysis and machine intelligence* 44, 7 (2021), 3496–3507.
- [7] Deyu Bo, Chuan Shi, Lele Wang, and Renjie Liao. 2023. Specformer: Spectral Graph Neural Networks Meet Transformers. In *The Eleventh International Conference on Learning Representations*.
- [8] Benjamin P Chamberlain, James Rowbottom, Davide Eynard, Francesco Di Giovanni, Xiaowen Dong, and Michael M Bronstein. 2021. Beltrami flow and neural diffusion on graphs. In *Proceedings of the 35th International Conference on Neural Information Processing Systems*. 1594–1609.
- [9] Hao Chen, Yuanchen Bei, Qijie Shen, Yue Xu, Sheng Zhou, Wenbing Huang, Feiran Huang, Senzhang Wang, and Xiao Huang. 2024. Macro graph neural networks for online billion-scale recommender systems. In *Proceedings of the ACM web conference 2024*. 3598–3608.
- [10] Minjie Cheng and Hongteng Xu. 2024. A Quasi-Wasserstein loss for learning graph neural networks. In *Proceedings of the ACM Web Conference 2024*. 815–826.
- [11] Zhiyong Cheng, Sai Han, Fan Liu, Lei Zhu, Zan Gao, and Yuxin Peng. 2023. Multi-behavior recommendation with cascading graph convolution networks. In *Proceedings of the ACM Web Conference 2023*. 1181–1189.
- [12] Eli Chien, Jianhao Peng, Pan Li, and Olgica Milenkovic. 2021. Adaptive Universal Generalized PageRank Graph Neural Network. In *International Conference on Learning Representations*.
- [13] Andreas Damianou, Francesco Fabbri, Paul Gigoli, Marco De Nadai, Alice Wang, Enrico Palumbo, and Mounia Lalmas. 2024. Towards graph foundation models for personalization. In *Companion Proceedings of the ACM Web Conference 2024*. 1798–1802.
- [14] Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. 2016. Convolutional neural networks on graphs with fast localized spectral filtering. In *Proceedings of the 30th International Conference on Neural Information Processing Systems*. 3844–3852.
- [15] Reinhard Diestel. 2025. *Graph theory*. Vol. 173. Springer Nature.
- [16] Lun Du, Xiaozhou Shi, Qiang Fu, Xiaojun Ma, Hengyu Liu, Shi Han, and Dongmei Zhang. 2022. Gbk-gnn: Gated bi-kernel graph neural networks for modeling both homophily and heterophily. In *Proceedings of the ACM web conference 2022*. 1550–1558.
- [17] Montacer Essid and Justin Solomon. 2018. Quadratically regularized optimal transport on graphs. *SIAM Journal on Scientific Computing* 40, 4 (2018), A1961–A1986.
- [18] Enrico Facca and Michele Benzi. 2021. Fast iterative solution of the optimal transport problem on graphs. *SIAM Journal on Scientific Computing* 43, 3 (2021), A2295–A2319.
- [19] Wenqi Fan, Yao Ma, Qing Li, Yuan He, Eric Zhao, Jiliang Tang, and Dawei Yin. 2019. Graph neural networks for social recommendation. In *The world wide web conference*. 417–426.
- [20] Aosong Feng, Chenyu You, Shiqiang Wang, and Leandros Tassioulas. 2022. Kergnns: Interpretable graph neural networks with graph kernels. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 36. 6614–6622.
- [21] Xinyu Fu, Jiani Zhang, Ziqiao Meng, and Irwin King. 2020. Magnn: Metapath aggregated graph neural network for heterogeneous graph embedding. In *Proceedings of The Web Conference 2020*. 2331–2341.
- [22] Chen Gao, Xiang Wang, Xiangnan He, and Yong Li. 2022. Graph neural networks for recommender system. In *Proceedings of the fifteenth ACM international conference on web search and data mining*. 1623–1625.
- [23] Johannes Gasteiger, Aleksandar Bojchevski, and Stephan Günnemann. 2019. Predict then Propagate: Graph Neural Networks meet Personalized PageRank. In *International Conference on Learning Representations*.
- [24] Jingwei Guo, Kaizhu Huang, Xinpeng Yi, and Rui Zhang. 2023. Graph Neural Networks with Diverse Spectral Filtering. In *Proceedings of the ACM Web Conference 2023*. 306–316.
- [25] Yuhe Guo and Zhewei Wei. 2023. Clenshaw graph neural networks. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 614–625.
- [26] William L Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*. 1025–1035.
- [27] Mingguo He, Zhewei Wei, and Ji-Rong Wen. 2022. Convolutional neural networks on graphs with chebyshev approximation, revisited. *Advances in Neural Information Processing Systems* 35 (2022), 7264–7276.
- [28] Mingguo He, Zhewei Wei, Hongteng Xu, et al. 2021. Bernnet: Learning arbitrary graph spectral filters via bernstein approximation. *Advances in Neural Information Processing Systems* 34 (2021), 14239–14251.
- [29] Dongyeon Hwang, Hyunju Kim, Sunwoo Kim, and Kijung Shin. 2024. Flowformer: Empowering neural architecture encoding using a flow-aware graph transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 6128–6137.
- [30] Dejun Jiang, Zhenxing Wu, Chang-Yu Hsieh, Guangyong Chen, Ben Liao, Zhe Wang, Chao Shen, Dongsheng Cao, Jian Wu, and Tingjun Hou. 2021. Could graph neural networks learn better molecular representation for drug discovery? A comparison study of descriptor-based and graph-based models. *Journal of cheminformatics* 13, 1 (2021), 1–23.
- [31] Wei Ju, Junwei Yang, Meng Qu, Weiping Song, Jianhao Shen, and Ming Zhang. 2022. Kgnn: Harnessing kernel-based networks for semi-supervised graph classification. In *Proceedings of the fifteenth ACM international conference on web search and data mining*. 421–429.
- [32] Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).
- [33] Thomas N Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*.
- [34] Johannes Klicpera, Stefan Weissenberger, and Stephan Günnemann. 2019. Diffusion improves graph learning. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. 13366–13378.
- [35] Guohao Li, Matthias Müller, Bernard Ghanem, and Vladlen Koltun. 2021. Training graph neural networks with 1000 layers. In *International conference on machine learning*. PMLR, 6437–6449.
- [36] Mengzhang Li and Zhanxing Zhu. 2021. Spatial-temporal fusion graph neural networks for traffic flow forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 4189–4196.
- [37] Derek Lim, Felix Hohne, Xiuyu Li, Sijia Linda Huang, Vaishnavi Gupta, Omkar Bhalerao, and Ser Nam Lim. 2021. Large Scale Learning on Non-Homophilous Graphs: New Benchmarks and Strong Simple Methods. In *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan (Eds.), Vol. 34. Curran Associates, Inc., 20887–20902. https://proceedings.neurips.cc/paper_files/paper/2021/file/a816a80e4c1c56caa2eb4e1819cbb2f-Paper.pdf
- [38] Jenny Liu, Aviral Kumar, Jimmy Ba, Jamie Kiros, and Kevin Swersky. 2019. Graph normalizing flows. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. 13578–13588.
- [39] Jiahong Ma, Mingguo He, and Zhewei Wei. 2024. Polyformer: Scalable node-wise filters via polynomial graph transformer. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2118–2129.
- [40] Joe Marino, Yisong Yue, and Stephan Mandt. 2018. Iterative amortized inference. In *International Conference on Machine Learning*. PMLR, 3403–3412.
- [41] Julian McAuley, Christopher Targett, Qinfeng Shi, and Anton Van Den Hengel. 2015. Image-based recommendations on styles and substitutes. In *Proceedings of the 38th international ACM SIGIR conference on research and development in information retrieval*. 43–52.
- [42] Mathias Niepert, Mohamed Ahmed, and Konstantin Kutzkov. 2016. Learning convolutional neural networks for graphs. In *International conference on machine learning*. PMLR, 2014–2023.
- [43] Namyoung Park, Andrey Kan, Xin Luna Dong, Tong Zhao, and Christos Faloutsos. 2019. Estimating node importance in knowledge graphs using graph neural networks. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 596–606.
- [44] Hongbin Pei, Bingzhe Wei, Kevin Chen-Chuan Chang, Yu Lei, and Bo Yang. 2020. Geom-GCN: Geometric Graph Convolutional Networks. In *International Conference on Learning Representations*.
- [45] Jiezhong Qiu, Jian Tang, Hao Ma, Yuxiao Dong, Kuansan Wang, and Jie Tang. 2018. Deepinf: Social influence prediction with deep learning. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 2110–2119.
- [46] Benedek Rozemberczki, Carl Allen, and Rik Sarkar. 2021. Multi-scale attributed node embedding. *Journal of Complex Networks* 9, 2 (2021), cnab014.
- [47] T Konstantin Rusch, Michael M Bronstein, and Siddhartha Mishra. 2023. A survey on oversmoothing in graph neural networks. *arXiv preprint arXiv:2303.10993* (2023).
- [48] Victor Garcia Satorras, Emiel Hoogeboom, and Max Welling. 2021. E (n) equivariant graph neural networks. In *International conference on machine learning*. PMLR, 9323–9332.

- [49] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Galligher, and Tina Eliassi-Rad. 2008. Collective classification in network data. *AI magazine* 29, 3 (2008), 93–93.
- [50] Rui Shu, Hung H Bui, Shengjia Zhao, Mykel J Kochenderfer, and Stefano Ermon. 2018. Amortized inference regularization. *Advances in Neural Information Processing Systems* 31 (2018).
- [51] Vayer Titouan, Nicolas Courty, Romain Tavenard, and Rémi Flamary. 2019. Optimal transport for structured data with application on graphs. In *International Conference on Machine Learning*. PMLR, 6275–6284.
- [52] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*. 6000–6010.
- [53] Petar Velicković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations*.
- [54] Cédric Vincent-Cuaz, Titouan Vayer, Rémi Flamary, Marco Corneli, and Nicolas Courty. 2021. Online graph dictionary learning. In *International Conference on Machine Learning*. PMLR, 10564–10574.
- [55] Huahua Wang and Arindam Banerjee. 2014. Bregman alternating direction method of multipliers. In *Proceedings of the 27th International Conference on Neural Information Processing Systems-Volume 2*. 2816–2824.
- [56] Hongwei Wang and Jure Leskovec. 2021. Combining graph convolutional neural networks and label propagation. *ACM Transactions on Information Systems (TOIS)* 40, 4 (2021), 1–27.
- [57] Lin Wang, Wenqi Fan, Jiatong Li, Yao Ma, and Qing Li. 2024. Fast graph condensation with structure-based neural tangent kernel. In *Proceedings of the ACM Web Conference 2024*. 4439–4448.
- [58] Xiao Wang, Houye Ji, Chuan Shi, Bai Wang, Yanfang Ye, Peng Cui, and Philip S Yu. 2019. Heterogeneous graph attention network. In *The world wide web conference*. 2022–2032.
- [59] Xiaoyang Wang, Yao Ma, Yiqi Wang, Wei Jin, Xin Wang, Jiliang Tang, Caiyan Jia, and Jian Yu. 2020. Traffic flow prediction via spatial temporal graph neural network. In *Proceedings of the web conference 2020*. 1082–1092.
- [60] Yuyang Wang, Jianren Wang, Zhonglin Cao, and Amir Barati Farimani. 2022. Molecular contrastive learning of representations via graph neural networks. *Nature Machine Intelligence* 4, 3 (2022), 279–287.
- [61] Zhihao Wen and Yuan Fang. 2022. Trend: Temporal event and node dynamics for graph representation learning. In *Proceedings of the ACM web conference 2022*. 1159–1169.
- [62] Felix Wu, Amauri Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Weinberger. 2019. Simplifying graph convolutional networks. In *International conference on machine learning*. PMLR, 6861–6871.
- [63] Qitian Wu, Wentao Zhao, Zenan Li, David Wipf, and Junchi Yan. 2022. NodeFormer: a scalable graph structure learning transformer for node classification. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*. 27387–27401.
- [64] Shu Wu, Zekun Li, Yanyue Su, Zeyu Cui, Xiaoyu Zhang, and Liang Wang. 2021. GraphFM: Graph factorization machines for feature interaction modeling. *arXiv preprint arXiv:2105.11866* (2021).
- [65] Bingbing Xu, HuaWei Shen, Qi Cao, Keting Cen, and Xueqi Cheng. 2019. Graph convolutional networks using heat kernel for semi-supervised learning. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence*. 1928–1934.
- [66] Hongteng Xu. 2020. Gromov-Wasserstein factorization models for graph clustering. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 34. 6478–6485.
- [67] Hongteng Xu, Jiachang Liu, Dixin Luo, and Lawrence Carin. 2022. Representing graphs via Gromov-Wasserstein factorization. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45, 1 (2022), 999–1016.
- [68] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How Powerful are Graph Neural Networks?. In *International Conference on Learning Representations*.
- [69] Zhilin Yang, William Cohen, and Ruslan Salakhudinov. 2016. Revisiting semi-supervised learning with graph embeddings. In *International conference on machine learning*. PMLR, 40–48.
- [70] Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. 2021. Do transformers really perform bad for graph representation?. In *Proceedings of the 35th International Conference on Neural Information Processing Systems*. 28877–28888.
- [71] Chaohao Yuan, Kangfei Zhao, Ercan Engin Kuruoglu, Liang Wang, Tingyang Xu, Wenbing Huang, Deli Zhao, Hong Cheng, and Yu Rong. 2025. A survey of graph transformers: Architectures, theories and applications. *arXiv preprint arXiv:2502.16533* (2025).
- [72] Shen Yuan, Shukai Gong, and Hongteng Xu. 2025. USPTO-LLM: A Large Language Model-Assisted Information-enriched Chemical Reaction Dataset. In *Companion Proceedings of the ACM on Web Conference 2025*. 817–820.

- [73] Jie Zhang and Min-Te Sun. 2024. From ChebNet to ChebGibbsNet. *arXiv preprint arXiv:2412.01789* (2024).
- [74] Zeyu Zhang, Jiamou Liu, Xianda Zheng, Yifei Wang, Pengqian Han, Yupan Wang, Kaiqi Zhao, and Zijian Zhang. 2023. RSGNN: A Model-agnostic Approach for Enhancing the Robustness of Signed Graph Neural Networks. In *Proceedings of the ACM Web Conference 2023*. 60–70.
- [75] Meiqi Zhu, Xiao Wang, Chuan Shi, Houye Ji, and Peng Cui. 2021. Interpreting and unifying graph neural networks with an optimization framework. In *Proceedings of the Web Conference 2021*. 1215–1226.
- [76] Chenyi Zhuang and Qiang Ma. 2018. Dual graph convolutional networks for graph-based semi-supervised classification. In *Proceedings of the 2018 world wide web conference*. 499–508.

A Detailed Derivation of Table 1

The derivation of the relations shown in Table 1 is based on i) the relation $\tilde{S}_V \tilde{S}_V^\top = \tilde{L} = I - \tilde{A}$ and ii) setting $Z = \tilde{S}_V^\top X$. Accordingly, the decomposition of GCN is straightforward:

$$(I + \tilde{A})X = (2I - \tilde{L})X = (2I - \tilde{S}_V \tilde{S}_V^\top)X = 2X - \tilde{S}_V Z. \quad (14)$$

For GPR-GNN, we have

$$\begin{aligned} & \sum_{m=0}^M \alpha_m \tilde{A}^m X \\ &= \sum_{m=0}^M \alpha_m (I - \tilde{L})^m X \\ &= \sum_{m=0}^M \alpha_m \left(\sum_{j=0}^m \binom{m}{j} (-1)^j \tilde{L}^j \right) X \\ &= \sum_{m=0}^M \alpha_m X + \sum_{m=1}^M \alpha_m \left(\sum_{j=1}^m \binom{m}{j} (-1)^j \tilde{L}^j \right) X \\ &= \sum_{m=0}^M \alpha_m X + \sum_{m=1}^M \alpha_m \left(\sum_{j=1}^m \binom{m}{j} (-1)^j \tilde{L}^{j-1} \tilde{S}_V \right) Z. \end{aligned} \quad (15)$$

Similarly, for BernNet, we have

$$\begin{aligned} & \sum_{m=0}^M \frac{\alpha_m}{2^M} \binom{M}{m} (2I - \tilde{L})^{M-m} \tilde{L}^m X \\ &= \frac{\alpha_0}{2^M} (2I - \tilde{L})^M X + \sum_{m=1}^M \frac{\alpha_m}{2^M} \binom{M}{m} (2I - \tilde{L})^{M-m} \tilde{L}^m X \\ &= \frac{\alpha_0}{2^M} (I + \tilde{A})^M X + \sum_{m=1}^M \frac{\alpha_m}{2^M} \binom{M}{m} (2I - \tilde{L})^{M-m} \tilde{L}^{m-1} \tilde{S}_V Z. \end{aligned} \quad (16)$$

For the above two equations, replacing $\tilde{L}$ with $\tilde{S}_V \tilde{S}_V^\top$ leads to the formulations in Table 1.

B More Experimental Results

B.0.1 Robustness to Hyperparameters. The weight of the transport cost term, i.e., β , is the key hyperparameter impacting the performance of our method. Empirically, when β is too large, the transport cost is penalized significantly, and the impact of AF module becomes weak in both the learning and prediction phases. As a result, GNN+AF may perform similarly to the original GNN when using a large β . On the contrary, when β is too small, the flow and the transport cost between the observed labels and their predictions are regularized weakly, which may lead to the over-fitting issue. We test the robustness of our method based on λ as defined in (11), where $\lambda = \frac{1}{\beta}$ and show representative results in Figure 5.

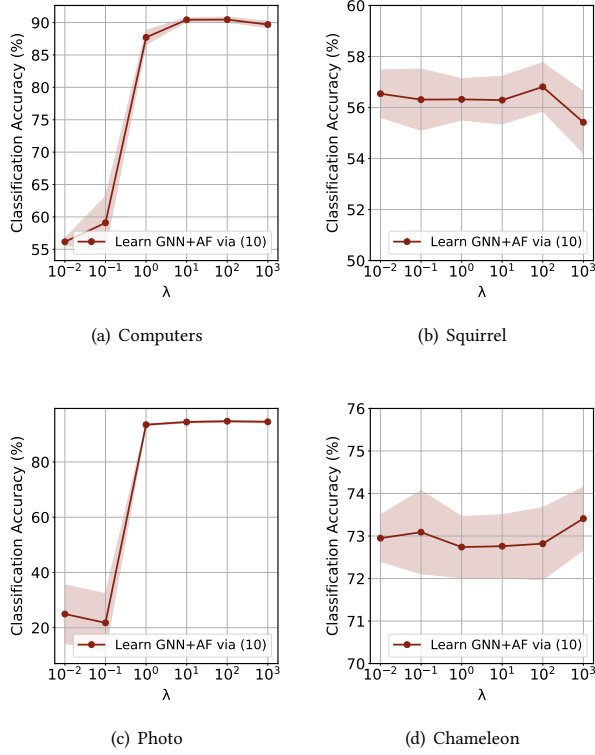


Figure 5: The impact of λ on the learning results.

We train ChebNetII+AF on two graphs with $\lambda \in [10^{-2}, 10^3]$. For the homophilic graph “Computers” and “Photo”, our model achieves stable performance when $\lambda \geq 1$. When $\lambda < 1$, the learning results degrade significantly because of inadequate supervision. On the contrary, our model often obtains stable results for the heterophilic graph “Squirrel” and “Chameleon” when $\lambda \in [10^{-2}, 10^2]$. These results demonstrate that we can set λ in a wide range to obtain relatively stable performance.

B.0.2 More Comparisons. Regarding the analysis of the distribution of learned flow and robustness to label insufficiency issue, besides conducting analytical experiments on Computers and Squirrel, we also consider Photo and Chameleon as the test graphs. Figure 6 shows the histograms of learned flows on Photo and Chameleon, respectively. Figure 7 shows the impact of label insufficiency on the model performance.

C Hyperparameter Settings

For all GNN models, we refer to BernNet [28], ChebNetII [27] and QWloss [10], adopt grid hyperparameter search to obtain the optimal results. Among the hyperparameters, the learning rate is set to $\{0.001, 0.002, 0.01, 0.05\}$, and the weight decay is set to $\{0.0, 0.005\}$. For our method, the range of λ is $\{0.01, 0.1, 1, 10, 100, 1000\}$. For PolyFormer [39], we use the same data and settings as those in its original paper.

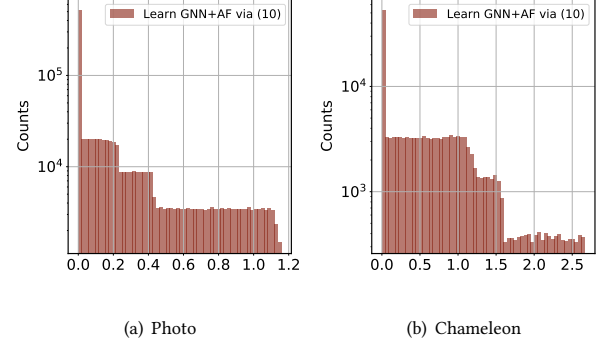


Figure 6: The histogram of learned flows for different graphs.

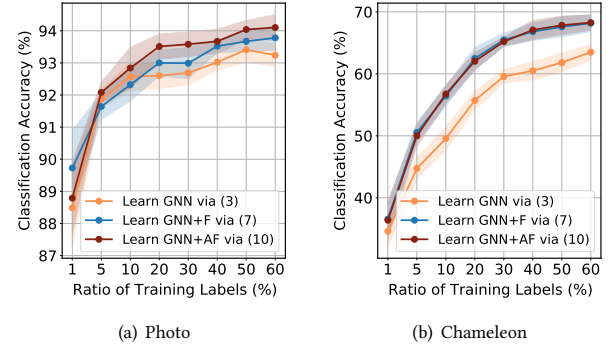


Figure 7: Illustrations of the learning methods' performance given different amounts of labeled nodes.